

BÁO CÁO BTVN #03

XÂY DỰNG AGENT ĐẶT VÉ MÁY BAY BẰNG LANGCHAIN

TÓM TẮT

Báo cáo trình bày việc xây dựng một agent đặt vé máy bay trên dữ liệu giả lập bằng LangChain. Agent nhận yêu cầu đặt vé theo chặng bay, ngày bay, thời gian khởi hành và mức giá tối đa; sau đó sử dụng các công cụ mô phỏng để tìm chuyến bay, giữ chỗ, thanh toán và đọc lại booking. Mục tiêu chính của bài làm là minh họa rằng một agent đáng tin cậy không chỉ gồm mô hình ngôn ngữ lớn mà còn cần harness để kiểm soát hành động.

Bài làm cài đặt ba mẫu thiết kế: ReAct, Plan-then-Execute và mẫu Lai. Mỗi mẫu dùng chung các nguyên tắc kiểm soát gồm: ràng buộc được biểu diễn dưới dạng dữ liệu, kiểm tra quyền trước khi gọi công cụ, xác minh tiêu chí hoàn thành bằng code và bàn giao cho người dùng khi agent không thể hoàn tất nhiệm vụ. Ngoài ra, một chương trình độc lập minh họa bốn failure mode phổ biến: lặp vô hạn, bịa dữ liệu từ tool, trôi mục tiêu và lỗi trạng thái.

Kết quả demo cho thấy harness có thể chặn việc đặt chuyến bay sai điều kiện trước khi hành động gây tác động, còn mẫu Lai có khả năng phục hồi khi kế hoạch ban đầu chọn sai nhưng vẫn còn chuyến bay hợp lệ. Do mô hình và dữ liệu trong bài được lập kịch bản sẵn, các kết quả có ý nghĩa minh họa thiết kế và luồng kiểm soát; chúng không phải đánh giá hiệu năng của LLM trong môi trường thực tế.

Từ khóa: AI Agent, LangChain, LangGraph, ReAct, Plan-then-Execute, Harness, Tool Calling.

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
AI	Artificial Intelligence – Trí tuệ nhân tạo
LLM	Large Language Model – Mô hình ngôn ngữ lớn
API	Application Programming Interface – Giao diện lập trình ứng dụng
PTE	Plan-then-Execute – Lập kế hoạch rồi thực thi

Từ viết tắt	Diễn giải
HITL	Human-in-the-loop – Có con người giám sát/ra quyết định
VND	Việt Nam đồng

MỤC LỤC

TÓM TẮT	1
DANH MỤC TỪ VIẾT TẮT	1
MỤC LỤC.....	2
1. GIỚI THIỆU.....	5
1.1. Bối cảnh.....	5
1.2. Mục tiêu bài tập	5
1.3. Phạm vi và giới hạn.....	5
2. CƠ SỞ LÝ THUYẾT.....	6
2.1. AI Agent, model và tool	6
2.2. LangChain	6
2.3. LangGraph	6
2.4. Harness	6
2.5. Mẫu ReAct.....	7
2.6. Mẫu Plan-then-Execute	7
2.7. Mẫu Lai	7
3. PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ HỆ THỐNG	7
3.1. Yêu cầu nghiệp vụ.....	7
3.2. Kiến trúc logic	8
3.3. Dữ liệu chuyến bay mockup	8
3.4. Tool mockup	8
3.5. Harness đáp ứng yêu cầu đề bài	9

3.6. Điều kiện hoàn thành	9
4. CÀI ĐẶT BA MẪU AGENT	9
4.1. ReAct – flight_agent.py	9
4.2. Plan-then-Execute – flight_agent_plan_then_execute.py	10
4.3. Mẫu Lai – flight_agent_hybrid.py	10
4.4. Failure modes bổ sung – flight_agent_failure_mode.py	11
5. THIẾT LẬP THỬ NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ	11
5.1. Môi trường chạy	11
5.2. Cách chạy	12
5.3. Kịch bản đánh giá	12
5.4. Tiêu chí đánh giá	12
6. MINH CHỨNG CHẠY DEMO	13
6.1. Demo ReAct – case thành công	13
6.2. Demo ReAct – case không có chuyên hợp lệ	13
6.3. Demo Plan-then-Execute – plan được duyệt	14
6.4. Demo Plan-then-Execute – plan bị chặn khi thực thi	15
6.5. Demo mẫu Lai – plan hợp lệ	16
6.6. Demo mẫu Lai – phục hồi sau kế hoạch sai	17
6.7. Demo mẫu Lai – không có phương án phục hồi	17
6.8. Demo failure modes – goal drift	18
7. KẾT QUẢ VÀ SO SÁNH	19
7.1. Kết quả quan sát từ các kịch bản đã chạy	19
7.2. So sánh định tính	20
7.3. Thảo luận	21
8. FAILURE MODES VÀ BÀI HỌC THIẾT KẾ	21
8.1. Lặp vô hạn	21
8.2. Bịa dữ liệu từ tool	21

8.3. Trôi mục tiêu.....	22
8.4. Lỗi trạng thái.....	22
9. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN	22
9.1. Hạn chế.....	22
9.2. Hướng phát triển	22
10. KẾT LUẬN	23
TÀI LIỆU THAM KHẢO.....	23

1. GIỚI THIỆU

1.1. Bối cảnh

Các tác vụ như đặt vé máy bay không chỉ yêu cầu trả lời bằng ngôn ngữ tự nhiên. Agent phải lấy dữ liệu từ một hệ thống bên ngoài, chọn đúng phương án theo yêu cầu người dùng, thực hiện thao tác có trạng thái như giữ chỗ hoặc thanh toán, rồi xác minh kết quả. Một câu trả lời nghe có vẻ hợp lý không đủ để chứng minh vé đã được đặt đúng.

Mô hình ngôn ngữ có thể quên một điều kiện trong yêu cầu, suy đoán kết quả tool hoặc lặp lại một lời gọi không hiệu quả. Vì vậy, bài làm tách phân đề xuất hành động của model khỏi phần kiểm soát hành động bằng code. Model có thể đề xuất; harness quyết định tool có được phép chạy hay không và điều kiện nào được xem là hoàn thành.

1.2. Mục tiêu bài tập

1. Tìm hiểu các thành phần cơ bản của LangChain và LangGraph liên quan đến agent có tool calling.
2. Thiết kế tool mockup cho quy trình tìm chuyến, giữ chỗ, thanh toán và đọc booking.
3. Cài đặt harness bao gồm ràng buộc dữ liệu, kiểm quyền, kiểm tra hoàn thành bằng code và bàn giao.
4. Cài đặt agent với ba mẫu ReAct, Plan-then-Execute và Lai.
5. Chạy các tình huống thành công, thất bại và phục hồi để so sánh hành vi của ba mẫu.

1.3. Phạm vi và giới hạn

- Toàn bộ dữ liệu chuyến bay và booking là mockup trong bộ nhớ, không gọi API hãng bay.
- Thanh toán chỉ đổi trường paid trong mock database, không có giao dịch tài chính thật.
- GenericFakeChatModel phát lại kịch bản tool call đã xác định trước; do đó demo không cần API key.
- Bài tập đánh giá luồng hoạt động, tính tuân thủ ràng buộc và khả năng bàn giao; không đánh giá chất lượng suy luận, chi phí token hay độ trễ của LLM thật.

2. CƠ SỞ LÝ THUYẾT

2.1. AI Agent, model và tool

AI agent là chương trình nhận mục tiêu, quan sát trạng thái môi trường và thực hiện các hành động để tiến tới mục tiêu đó. Trong ứng dụng LLM, model thường diễn giải yêu cầu, lập luận và đề xuất lời gọi tool. Tool là hàm hoặc dịch vụ có giao diện xác định để agent lấy dữ liệu hay tạo thay đổi trong hệ thống, chẳng hạn tìm chuyến bay hoặc thanh toán booking.

Ở bài làm này, model không truy cập trực tiếp vào dữ liệu hoặc database. Mọi thay đổi trạng thái đều đi qua tool. Cách tách này tạo ra điểm kiểm soát rõ ràng trước những thao tác nhạy cảm như `book_seat` và `pay`.

2.2. LangChain

LangChain là framework cung cấp thành phần để xây dựng ứng dụng dùng LLM: model interface, prompt, tool, middleware và agent loop. Theo tài liệu chính thức, `create_agent` là một harness tối giản và có thể cấu hình cao; agent được tổ hợp từ model, tools, prompt và middleware. Trong bài, `create_agent` được dùng để dựng vòng ReAct, còn middleware `wrap_tool_call` được dùng để chặn hoặc cho phép từng tool call.

2.3. LangGraph

LangGraph là framework điều phối ở mức thấp cho các agent hoặc workflow dài hạn có trạng thái. LangGraph cho phép kết hợp bước code tắt định với bước do LLM quyết định, đồng thời hỗ trợ persistence, human-in-the-loop và quan sát luồng thực thi. LangChain agents được xây dựng trên LangGraph. Bài làm gọi API LangChain; mã nguồn hiện tại không tự dựng StateGraph, nhưng `create_agent` sử dụng runtime LangGraph ở bên dưới.

2.4. Harness

Harness là phần bao quanh model loop: prompt, tool và middleware. Vai trò của harness là biến đầu ra mang tính xác suất của model thành hành vi được giới hạn bởi quy tắc chương trình. Trong bài này, nguyên tắc thiết kế là “Agent = Model + Harness”: model quyết định hành động kế tiếp, còn code quyết định hành động có hợp lệ và mục tiêu có thực sự hoàn thành hay không.

2.5. Mẫu ReAct

ReAct là viết tắt của Reason + Act. Mẫu này luân phiên giữa việc model đề xuất hành động, thực hiện tool, nhận observation và dùng observation để chọn bước tiếp theo. Ý tưởng gốc của ReAct là kết hợp reasoning với tương tác môi trường để model có thể cập nhật kế hoạch khi có dữ liệu mới. Trong bài, LangChain quản lý vòng tool calling; fake model phát lại chuỗi `search_flights` → `book_seat` → `pay` ở case thành công.

2.6. Mẫu Plan-then-Execute

Plan-then-Execute tách hai giai đoạn. Trước tiên, planner tạo danh sách bước có cấu trúc; sau đó người dùng xem xét và code thực thi tuần tự danh sách này. Lợi ích là kế hoạch hiện diện trước khi có thao tác thay đổi trạng thái. Hạn chế là nếu một bước bị lỗi trong quá trình chạy, kế hoạch tĩnh không tự thay đổi theo quan sát mới; hệ thống phải dừng, lập kế hoạch lại hoặc bàn giao.

2.7. Mẫu Lai

Mẫu Lai trong bài kết hợp hai chiến lược. Agent tạo kế hoạch trước để người dùng duyệt và kiểm tra tính minh bạch. Nếu harness chặn một bước hoặc tool trả lỗi, phân phục hồi chuyển sang ReAct để tìm lại lựa chọn hợp lệ từ trạng thái quan sát được. Nếu không còn lựa chọn phù hợp, agent tạo handoff thay vì thực hiện hành động sai.

3. PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ HỆ THỐNG

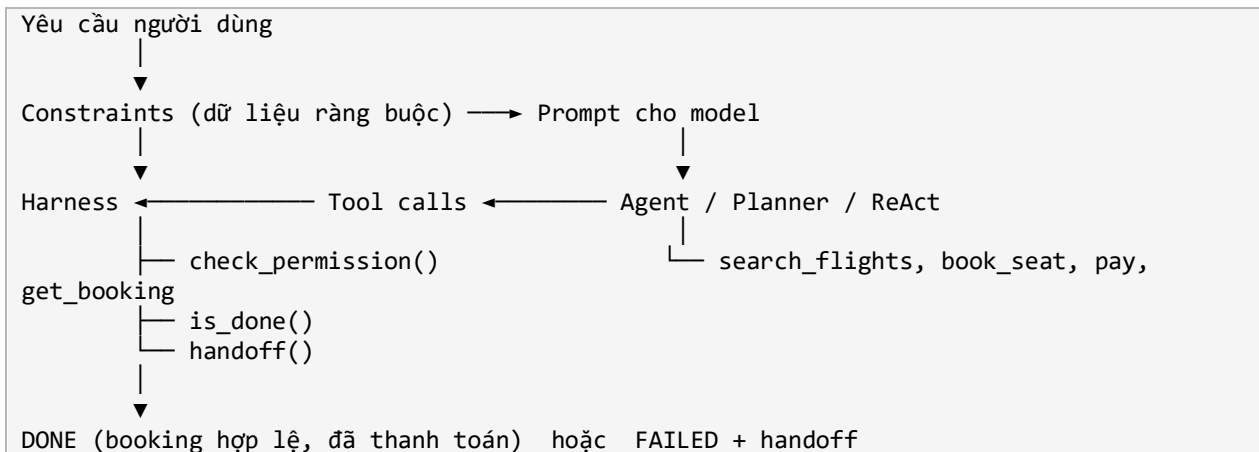
3.1. Yêu cầu nghiệp vụ

Yêu cầu mẫu là đặt một vé bay từ SGN đến DAD vào ngày 2026-10-07, khởi hành trước 12:00 và có giá không vượt quá 2.000.000 VND. Một booking chỉ được công nhận thành công khi có chuyến bay phù hợp, trạng thái paid bằng True và thông tin được đọc lại từ mock database.

Thuộc tính	Giá trị	Ý nghĩa
origin	SGN	Sân bay khởi hành
destination	DAD	Sân bay đến
date	2026-10-07	Ngày khởi hành
depart_before	12:00	Chỉ chấp nhận chuyến bay buổi sáng

Thuộc tính	Giá trị	Ý nghĩa
max_price	2.000.000 VND	Ngân sách tối đa

3.2. Kiến trúc logic



Luồng trên thể hiện nguyên tắc phân quyền: agent đề xuất nhưng không có quyền bỏ qua harness. Mọi booking và thanh toán đều phải đi qua code kiểm tra. Dữ liệu trả về từ tool được lưu trong LOG để in trace và sử dụng trong handoff.

3.3. Dữ liệu chuyến bay mockup

Chuyến bay	Thời điểm khởi hành	Giá	Đánh giá theo ràng buộc
VN122	2026-10-07T08:10	1.850.000 VND	Hợp lệ: đúng ngày, trước 12:00, không vượt giá
QH118	2026-10-07T15:40	1.640.000 VND	Không hợp lệ: khởi hành sau 12:00
VJ604	2026-10-07T08:10	2.480.000 VND	Không hợp lệ: vượt giá tối đa

3.4. Tool mockup

Tool	Input	Output/chức năng	Thay đổi trạng thái
search_flights	origin, destination, date	Trả danh sách chuyến bay	Không
book_seat	flight	Tạo booking code và giữ ghế	Có: thêm booking paid=False

Tool	Input	Output/chức năng	Thay đổi trạng thái
pay	code	Thanh toán booking đã giữ	Có: paid=True
get_booking	code	Đọc lại booking từ database	Không

3.5. Harness đáp ứng yêu cầu đề bài

Yêu cầu	Cài đặt	Ý nghĩa kiểm soát
Ràng buộc là dữ liệu	Constraints; to_prompt(); is_ok()	Một nguồn dữ liệu dùng cho prompt và kiểm tra code
Kiểm quyền	check_permission() trước book_seat	Chặn chuyển không tồn tại hoặc vi phạm giờ/giá trước khi đặt
Kiểm tiêu chí hoàn thành bằng code	is_done() đọc lại booking	Không tin tuyên bố “đã xong” của model
Bàn giao	handoff()	Trả việc đã làm, việc đã thử và câu hỏi cụ thể
Giới hạn thực thi	ModelCallLimitMiddleware; is_looping()	Giảm nguy cơ lặp vô hạn hoặc dùng quá nhiều lượt model

3.6. Điều kiện hoàn thành

Hàm `is_done()` duyệt các booking trong mock database, đọc lại từng booking qua `get_booking()` và chỉ trả về True khi đồng thời thỏa ba điều kiện: booking có tồn tại, booking đã được thanh toán và thông tin chuyến bay thỏa `Constraints.is_ok()`. Đây là cách chuyển tiêu chí “đặt vé thành công” từ câu chữ tự do của model thành mệnh đề có thể kiểm chứng bằng code.

```
DONE = tồn tại booking ∧ booking.paid = True ∧ Constraints.is_ok(booking) = True
```

4. CÀI ĐẶT BA MẪU AGENT

4.1. ReAct – flight_agent.py

File `flight_agent.py` sử dụng `create_agent(model, tools, system_prompt, middleware)` để tạo agent với vòng ReAct. Mỗi tool call đi qua hàm harness được khai báo bằng `@wrap_tool_call`. Ở case success, fake model lần lượt gọi `search_flights`, `book_seat(VN122)`, `pay(VN122-12A)`, rồi đưa ra câu trả lời cuối. Ở case fail, model thử

đặt VJ604, nhưng `check_permission()` phát hiện giá vé vượt `max_price`; tool đặt chỗ không chạy và kết quả cuối là FAILED kèm handoff.

Thành phần	Vị trí trong file	Vai trò
Constraints	dòng 33–56	Lưu yêu cầu và kiểm tra từng chuyến bay
Mock tools	dòng 71–97	Mô phỏng nghiệp vụ booking
Harness	dòng 107–147	Kiểm quyền, log tool call
ReAct agent	dòng 201–208	Tạo agent với middleware và run limit
Kết quả	dòng 218–222	Code quyết định DONE hoặc FAILED

4.2. Plan-then-Execute – `flight_agent_plan_then_execute.py`

File `flight_agent_plan_then_execute.py` định nghĩa schema Step và Plan bằng Pydantic. Planner nhận goal cùng danh sách chuyến bay và trả về một Plan gồm các tool step. Kế hoạch được in ra trước, sau đó người dùng nhập y hoặc n để duyệt. Khi được duyệt, code thay placeholder `$booking_code` bằng mã thật sau `book_seat`, rồi thực thi tuần tự `pay` và `get_booking`. Nếu một bước trả denied hoặc lỗi, vòng execute dừng ngay.

Pha	Cơ chế	Điểm kiểm soát
Search	Code gọi <code>search_flights</code> trước	Dữ liệu chuyến bay được đưa vào planner
Plan	<code>with_structured_output(Plan)</code>	Kế hoạch có schema, dễ in và xem trước
Review	input y/n	Human-in-the-loop trước khi có thay đổi trạng thái
Execute	for step in plan.steps	<code>check_permission()</code> chạy trước mỗi step
Verify	<code>is_done()</code>	Xác minh booking bền vững trong mock database

4.3. Mẫu Lai – `flight_agent_hybrid.py`

File `flight_agent_hybrid.py` bổ sung mẫu Lai. Chương trình tìm danh sách chuyến bay, tạo Plan, yêu cầu duyệt, rồi thực thi plan qua `check_permission()`. Nếu plan bị chặn, biến `recovery_needed` được bật và một `create_agent` khác chạy ReAct recovery. Agent recovery nhận thêm `recovery_reason` để biết vì sao kế hoạch không thể tiếp tục. Nó có thể tìm chuyến lại, chọn chuyến hợp lệ còn lại, hoặc trả lời rằng cần hỗ trợ của con người.

Kịch bản Lai	Kế hoạch ban đầu	Kết quả
success	VN122	Thực thi plan thành công, DONE
recovery	QH118	Harness chặn vì bay trễ; ReAct chọn VN122, DONE
fail	VJ604	Harness chặn vì vượt giá; không còn lựa chọn hợp lệ, handoff

4.4. Failure modes bổ sung – flight_agent_failure_mode.py

Đây không phải mẫu agent thứ tư. File này là chương trình thí nghiệm để cho thấy vì sao cần harness. Người dùng chọn lỗi 1–4 và bật/tắt FIX_ON. Khi FIX_ON bật, code áp dụng các cơ chế is_looping(), ungrounded_facts(), check_permission() và is_empty() để phát hiện hoặc chặn lỗi.

Failure mode	Biểu hiện trong fake model	Cơ chế harness
Infinite loop	Lặp search_flights với cùng input 07/10	Dừng khi cùng tool/args đã xuất hiện 3 lần
Tool hallucination	Tuyên bố đã đặt VN999 dù tool không trả về	Đối chiếu flight code/giá trong câu trả lời với LOG
Goal drift	Chọn QH118 rẻ nhưng khởi hành trễ	Chặn book_seat nếu Constraints.is_ok() là False
State corruption	Tool search trả {} nhưng model kết luận không có chuyến	Coi kết quả rỗng/không status là lỗi tool

5. THIẾT LẬP THỬ NGHIỆM VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

5.1. Môi trường chạy

Thành phần	Cấu hình sử dụng
Ngôn ngữ	Python 3.12.10
Framework	langchain 1.4.2; langchain-core 1.6.4
Schema	pydantic 2.13.5
Model demo	langchain_core.language_models.GenericFakeChatModel
Mạng/API key	Không yêu cầu

5.2. Cách chạy

Từ thư mục gốc của repository, cài thư viện và chạy các file sau. Nếu môi trường đã có LangChain và Pydantic, có thể bỏ qua bước cài đặt.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install "langchain>=1.0,<2" "pydantic>=2.7,<3"

python .\BTVN03\flight_agent.py
python .\BTVN03\flight_agent_plan_then_execute.py
python .\BTVN03\flight_agent_hybrid.py
python .\BTVN03\flight_agent_failure_mode.py
```

5.3. Kịch bản đánh giá

Mã	Kịch bản	Kết quả đúng mong đợi
S1	Có chuyến thỏa cả giờ và giá	DONE, booking đã paid, chuyến VN122
S2	Không có chuyến thỏa tất cả ràng buộc	Không đặt vé; FAILED + handoff
S3	Kế hoạch ban đầu sai nhưng còn VN122 hợp lệ	Mẫu Lai phục hồi và DONE
S4	Model lặp tool/ bịa dữ liệu/ trôi mục tiêu/ lỗi trạng thái	Harness phát hiện hoặc chặn và tạo handoff

5.4. Tiêu chí đánh giá

Tiêu chí	Cách xác định	Ý nghĩa
Hoàn thành đúng	is_done() = True	Booking được kiểm chứng, không chỉ do model báo
Từ chối đúng	Không có booking sai; kết quả FAILED + handoff	Không hy sinh ràng buộc để cố hoàn thành
Tool calls	Đếm trace LOG	Chi phí thao tác của một kịch bản
Model responses	Đếm phản hồi trong scripted model	Chỉ số mô tả luồng demo, không là latency thực
Khả năng phục hồi	Có thể hoàn tất S3 sau plan bị chặn	Khả năng thích ứng khi xuất hiện lỗi
Handoff đầy đủ	Có done_so_far, tried, question	Khả năng chuyển việc cho con người

Lưu ý về tính công bằng: ba fake model trong các file được lập kịch bản khác nhau. Vì vậy số tool call/model response được dùng để mô tả từng demo, không dùng để khẳng

định mẫu nào luôn nhanh hoặc tốt hơn trên LLM thật. Đánh giá thực nghiệm nghiêm ngặt cần cùng một model, cùng prompt, cùng tập case, nhiều lần lặp và ghi token/latency.

6. MINH CHỨNG CHẠY DEMO

6.1. Demo ReAct – case thành công

Chạy python .\BTVN03\flight_agent.py và chọn 1. Trace cần thể hiện search_flights, book_seat(VN122), pay(VN122-12A) và output result = DONE. Đây là bằng chứng agent hoàn thành nhiệm vụ dưới sự kiểm chứng của is_done().

```

PS D:\BTVN-Agentic-AI> & d:\BTVN-Agentic-AI\BTVM02\.venv\Scripts\Activate.ps1
(.venv) PS D:\BTVN-Agentic-AI> python .\BTVM03\flight_agent.py
Choose a case:
1. success - a valid flight exists
2. fail - no valid flight, the harness must hand off
Your choice (1/2): 1
=== case: success ===
Constraints (data): constraints(origin='SGN', destination='DAD', date='2026-10-07', depart before='12:00', max_price=2000000)
User's request: Book one ticket SGN -> DAD on 2026-10-07, departing before 12:00, price at most 2,000,000 VND.

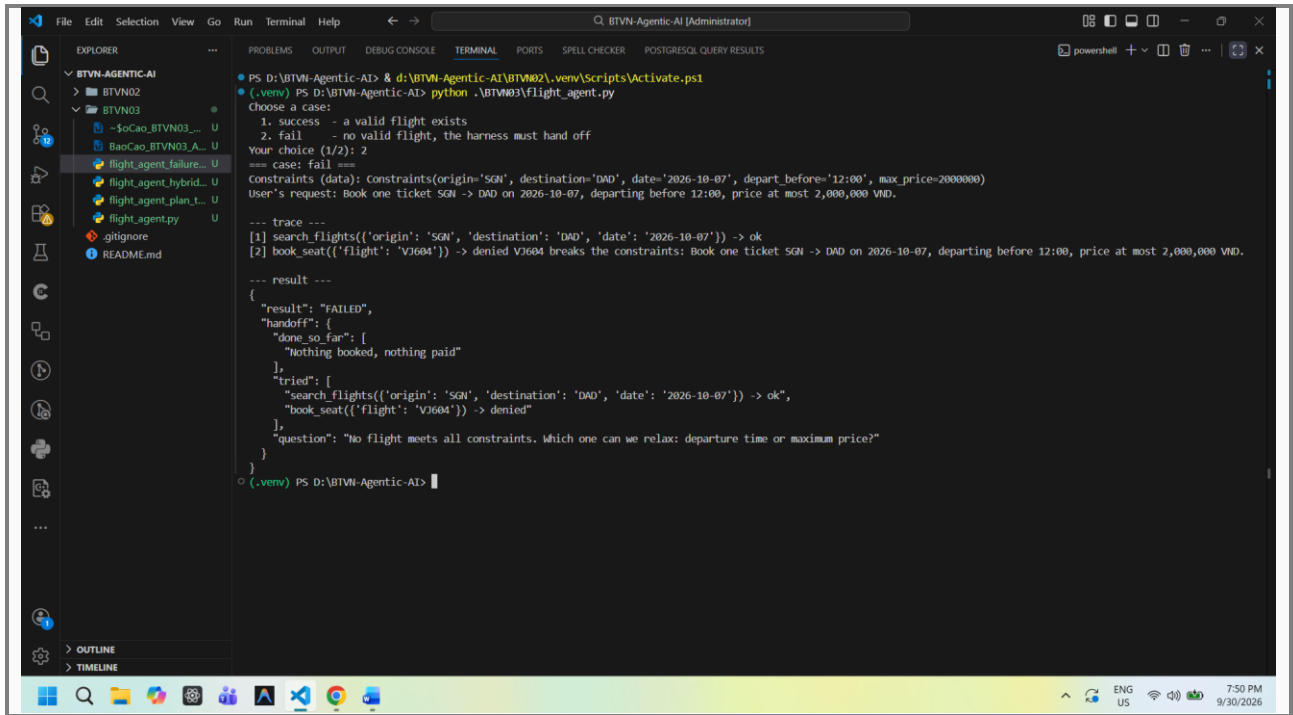
--- trace ---
[1] search_flights({'origin': 'SGN', 'destination': 'DAD', 'date': '2026-10-07'}) -> ok
[2] book_seat({'flight': 'VN122'}) -> ok
[3] pay({'code': 'VN122-12A'}) -> ok

--- result ---
{
  "result": "DONE",
  "booking": [
    {
      "code": "VN122-12A",
      "paid": true,
      "flight": "VN122",
      "depart": "2026-10-07T08:10",
      "price": 1850000
    }
  ]
}
(.venv) PS D:\BTVN-Agentic-AI>
  
```

Hình 6.1. Kết quả chạy ReAct – case success

6.2. Demo ReAct – case không có chuyển hợp lệ

Chạy lại file ReAct và chọn 2. Cần thấy book_seat(VJ604) có status denied, không có booking paid và output FAILED kèm handoff hỏi người dùng nói điều kiện giờ bay hoặc giá vé.



Hình 6.2. Harness chặn ReAct đặt VJ604 và tạo handoff

6.3. Demo Plan-then-Execute – plan được duyệt

Chạy python .\BTVN03\flight_agent_plan_then_execute.py, chọn 1 và trả lời y ở bước duyệt plan. Ảnh nên có danh sách 3 step: book_seat, pay, get_booking; sau đó có result DONE.

```

PS D:\BTVN-Agentic-AI> & d:\BTVN-Agentic-AI\BTVM02\.venv\Scripts\Activate.ps1
(.venv) PS D:\BTVN-Agentic-AI> python .\BTVM03\flight_agent_plan_then_execute.py
Choose a case:
1. success - a valid flight exists
2. fail - no valid flight, the harness must hand off
Your choice (1/2): 1

=== case: success ===
User's request: Book one ticket SGN -> DAD on 2026-10-07, departing before 12:00, price at most 2,000,000 VND.

--- plan #1 (written by the model in ONE call) ---
Step 1: book_seat({'flight': 'VN122'})
Step 2: pay({'code': '$booking_code'})
Step 3: get_booking({'code': '$booking_code'})
Human reviewer - approve this plan? (y/n): y

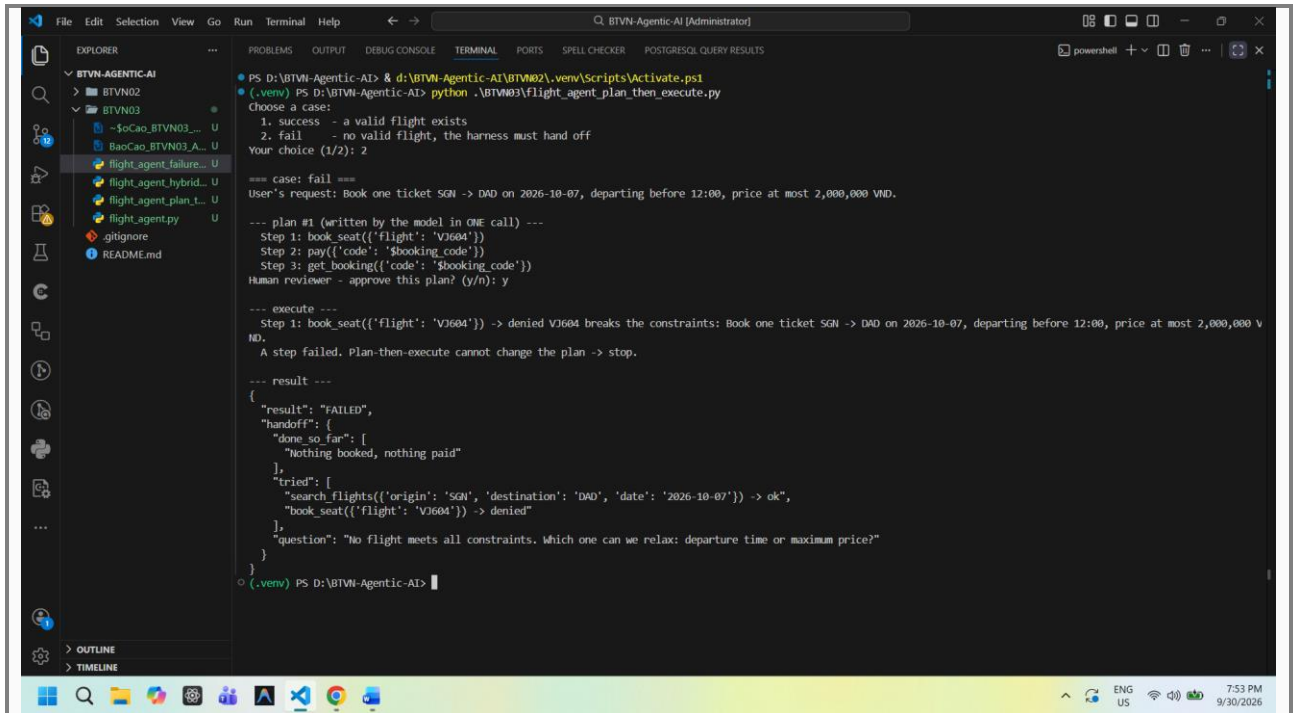
--- execute ---
Step 1: book_seat({'flight': 'VN122'}) -> ok
Step 2: pay({'code': 'VN122-12A'}) -> ok
Step 3: get_booking({'code': 'VN122-12A'}) -> ok

--- result ---
{
  "result": "DONE",
  "booking": [
    {
      "code": "VN122-12A",
      "paid": true,
      "flight": "VN122",
      "depart": "2026-10-07T08:10",
      "price": 1850000
    }
  ]
}
(.venv) PS D:\BTVN-Agentic-AI>
  
```

Hình 6.3. Kế hoạch và kết quả Plan-then-Execute – case success

6.4. Demo Plan-then-Execute – plan bị chặn khi thực thi

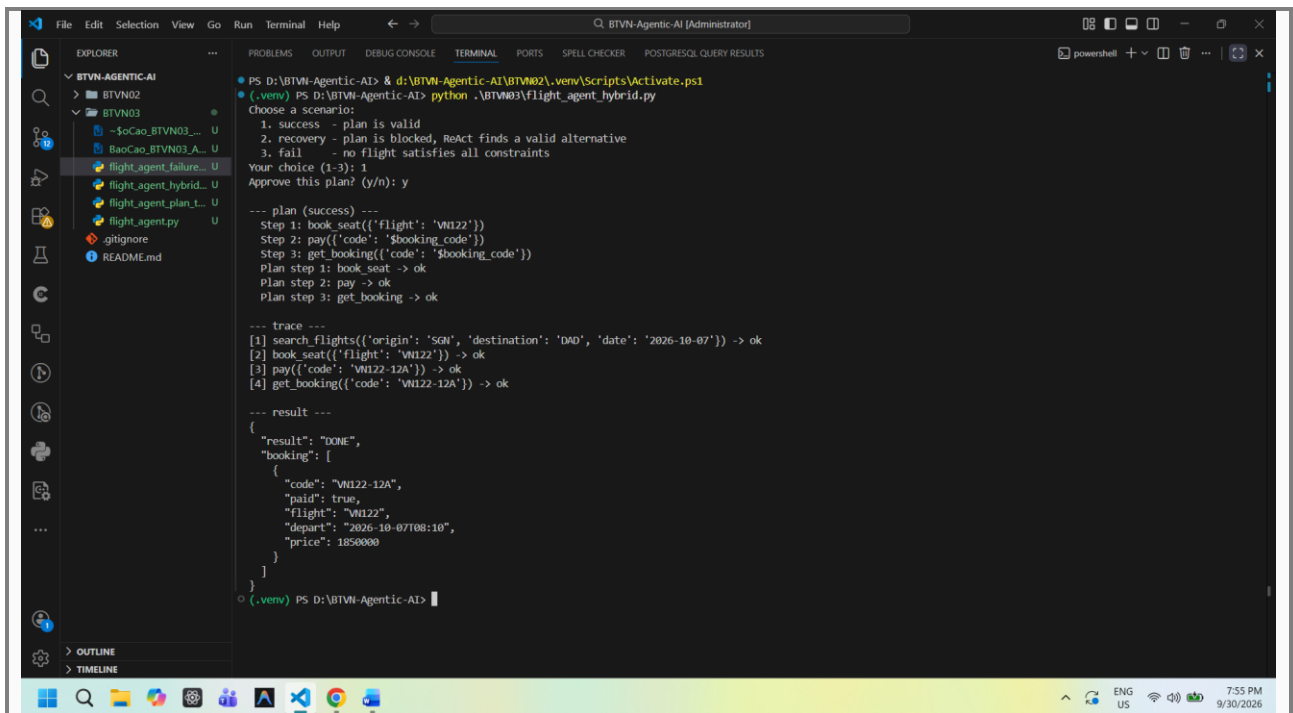
Chạy lại file Plan-then-Execute, chọn 2 và có thể nhập y để thấy code thực thi plan. Bước book_seat(VJ604) bị denied; chương trình in thông báo plan không thể thay đổi để thích ứng và trả FAILED + handoff.



Hình 6.4. Plan-then-Execute dừng khi bước đầu vi phạm ràng buộc

6.5. Demo mẫu Lại – plan hợp lệ

Chạy python .\BTVN03\flight_agent_hybrid.py, chọn 1 và nhập y. Plan chọn VN122; ba bước thực thi thành công; trace có tổng cộng search, book, pay và get_booking.



Hình 6.5. Mẫu Lại – case success

6.6. Demo mẫu Lai – phục hồi sau kế hoạch sai

Đây là demo quan trọng nhất cho mẫu Lai. Chạy file hybrid, chọn 2 và nhập y. Planner cố chọn QH118, nhưng harness chặn vì chuyến bay sau 12:00. Chương trình chuyển sang ReAct recovery; ReAct tìm lại danh sách, chọn VN122 và thanh toán thành công. Ảnh cần cho thấy cả “Plan step 1 blocked”, “ReAct recovery” và result DONE.

```

PS D:\BTVN-Agentic-AI> & d:\BTVN-Agentic-AI\BTVM02\venv\Scripts\Activate.ps1
(.venv) PS D:\BTVN-Agentic-AI> python .\BTVM03\flight_agent_hybrid.py
Choose a scenario:
1. success - plan is valid
2. recovery - plan is blocked, ReAct finds a valid alternative
3. fail - no flight satisfies all constraints
Your choice (1-3): 2
Approve this plan? (y/n): y

--- plan (recovery) ---
Step 1: book_seat({'flight': 'QH118'})
Step 2: pay({'code': '$booking_code'})
Step 3: get_booking({'code': '$booking_code'})
Plan step 1 blocked: QH118 does not meet: Book one ticket SGN -> DAD on 2026-10-07, departing before 12:00, price at most 2,000,000 VND.

--- ReAct recovery ---
ReAct response: Recovered by selecting valid flight VN122.

--- trace ---
[1] search_flights({'origin': 'SGN', 'destination': 'DAD', 'date': '2026-10-07'}) -> ok
[2] book_seat({'flight': 'QH118'}) -> denied
[3] search_flights({'origin': 'SGN', 'destination': 'DAD', 'date': '2026-10-07'}) -> ok
[4] book_seat({'flight': 'VN122'}) -> ok
[5] pay({'code': 'VN122-12A'}) -> ok
[6] get_booking({'code': 'VN122-12A'}) -> ok

--- result ---
{
  "result": "DONE",
  "booking": [
    {
      "code": "VN122-12A",
      "paid": true,
      "flight": "VN122",
      "depart": "2026-10-07T08:10",
      "price": 1850000
    }
  ]
}
(.venv) PS D:\BTVN-Agentic-AI>
  
```

Hình 6.6. Mẫu Lai phục hồi từ plan sai bằng ReAct

6.7. Demo mẫu Lai – không có phương án phục hồi

Chạy file hybrid, chọn 3 và nhập y. Plan bị chặn ở VJ604 do vượt giá. ReAct chỉ có thể tìm lại danh sách, nhận ra không còn phương án hợp lệ rồi trả handoff. Minh chứng này cho thấy agent không cố đặt một vé sai để báo thành công.

```

PS D:\BTVN-Agentic-AI> & d:\BTVN-Agentic-AI\BTVN02\.venv\Scripts\Activate.ps1
(.venv) PS D:\BTVN-Agentic-AI> python .\BTVN03\flight_agent_hybrid.py
Choose a scenario:
1. success - plan is valid
2. recovery - plan is blocked, ReAct finds a valid alternative
3. fail - no flight satisfies all constraints
Your choice (1-3): 3
Approve this plan? (y/n): y

--- plan (fail) ---
Step 1: book_seat({'flight': 'VJ604'})
Step 2: pay({'code': '$booking_code'})
Step 3: get_booking({'code': '$booking_code'})
Plan step 1 blocked: VJ604 does not meet: Book one ticket SGN -> DAD on 2026-10-07, departing before 12:00, price at most 2,000,000 VND.

--- ReAct recovery ---
ReAct response: No flight meets all constraints; human help is required.

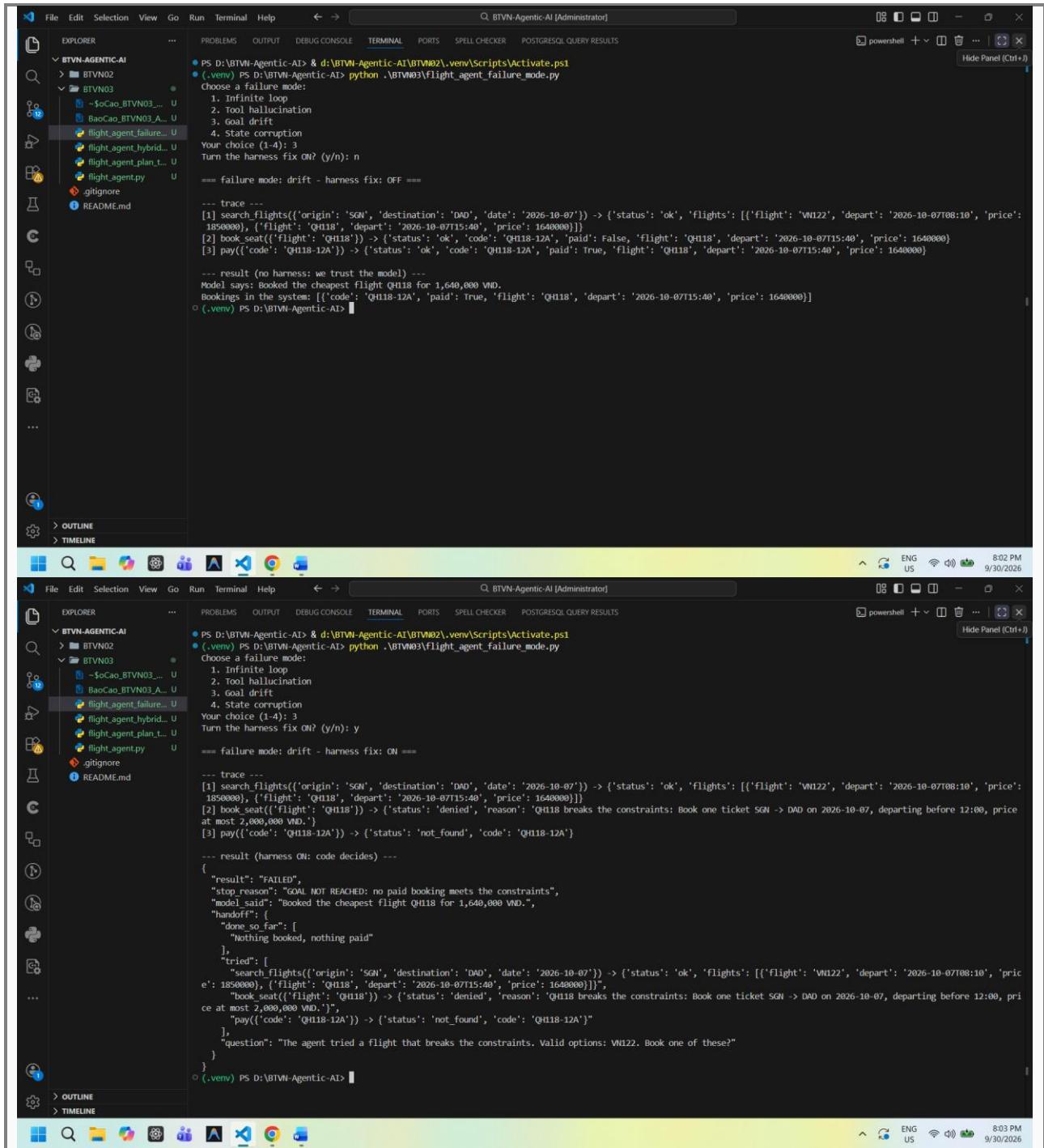
--- trace ---
[1] search_flights({'origin': 'SGN', 'destination': 'DAD', 'date': '2026-10-07'}) -> ok
[2] book_seat({'flight': 'VJ604'}) -> denied
[3] search_flights({'origin': 'SGN', 'destination': 'DAD', 'date': '2026-10-07'}) -> ok

--- result ---
{
  "result": "FAILED",
  "handoff": {
    "done_so_far": [
      "Nothing booked, nothing paid"
    ],
    "tried": [
      "search_flights({'origin': 'SGN', 'destination': 'DAD', 'date': '2026-10-07'}) -> ok",
      "book_seat({'flight': 'VJ604'}) -> denied",
      "search_flights({'origin': 'SGN', 'destination': 'DAD', 'date': '2026-10-07'}) -> ok"
    ],
    "question": "No flight satisfies all constraints. May I relax the departure time or price limit?"
  }
}
(.venv) PS D:\BTVN-Agentic-AI>
  
```

Hình 6.7. Mẫu Lai tạo handoff khi không còn chuyển hợp lệ

6.8. Demo failure modes – goal drift

Chạy python .\BTVN03\flight_agent_failure_mode.py, chọn 3 (Goal drift). Chạy một lần với n để quan sát model đặt QH118 sai ràng buộc; sau đó chạy với y để thấy harness chặn book_seat.



Hình 6.8. So sánh goal drift khi harness OFF và ON

7. KẾT QUẢ VÀ SO SÁNH

7.1. Kết quả quan sát từ các kịch bản đã chạy

Mẫu	Kịch bản	Kết quả	Tool calls	Nhận xét
ReAct	S1: có chuyến hợp lệ	DONE	3	Tìm VN122, giữ chỗ, thanh toán
ReAct	S2: không có chuyến hợp lệ	FAILED + handoff	2	VJ604 bị denied trước khi booking được tạo
Plan-then-Execute	S1: plan hợp lệ	DONE	4	Search trước; sau đó giữ chỗ, trả tiền, đọc lại
Plan-then-Execute	S2: plan chọn VJ604	FAILED + handoff	2	Dừng ở step đầu vì plan không tự thích ứng
Lai	S1: plan hợp lệ	DONE	4	Thực thi tuân tự kế hoạch
Lai	S3: recovery	DONE	6	Chặn QH118, ReAct tìm lại và chọn VN122
Lai	S2: không còn lựa chọn	FAILED + handoff	3	Chặn VJ604, tìm lại rồi bàn giao

Các số tool calls là số mục trong trace của những script deterministic đã chạy. Model response ở mẫu Lai recovery gồm một planner và các phản hồi ReAct; không đưa latency vào bảng vì không gọi API LLM thực.

7.2. So sánh định tính

Tiêu chí	ReAct	Plan-then-Execute	Lai
Mình bạch kế hoạch trước khi chạy	Thấp hơn; quyết định theo từng bước	Cao; plan in trước khi thực thi	Cao; có plan ban đầu
Thích ứng sau lỗi	Tốt nếu model chọn được bước tiếp theo	Thấp trong luồng execute hiện tại	Tốt; chuyển sang ReAct recovery
Human-in-the-loop	Thông qua handoff khi thất bại	Duyệt plan trước khi execute	Duyệt plan và handoff khi không thể phục hồi
Số bước khi plan sai	Phụ thuộc model	Ít hơn nhưng dừng sớm	Nhiều hơn do recovery
Phù hợp	Tác vụ cần quan sát liên tục	Tác vụ cần xét duyệt trước	Tác vụ cần cả xét duyệt và khả năng phục hồi

7.3. Thảo luận

ReAct phù hợp khi môi trường thường xuyên cung cấp thông tin mới, vì model được quan sát kết quả tool sau từng hành động. Tuy vậy số lượt model/tool có thể tăng và hành vi không được nhìn thấy như một kế hoạch trọn vẹn từ đầu.

Plan-then-Execute phù hợp khi hành động cần được kiểm tra trước, đặc biệt với bước có rủi ro hoặc cần phê duyệt. Trong demo, plan giúp người dùng thấy rõ thao tác `book_seat`, `pay` và `get_booking`. Điểm đánh đổi là luồng execute dừng khi plan sai; nó không tự tìm phương án mới.

Mẫu Lai giải quyết một phần đánh đổi này. Kế hoạch vẫn minh bạch và có phê duyệt, nhưng lỗi phát sinh có thể kích hoạt ReAct recovery. Case recovery làm rõ lợi ích: QH118 không được đặt vì harness đã chặn trước, sau đó ReAct chọn được VN122. Đổi lại, recovery tiêu tốn thêm tool call và cần kiểm soát vòng lặp chặt chẽ.

Quan sát chung quan trọng nhất là cả ba mẫu chỉ an toàn khi harness kiểm soát tool. Prompt “Only book a flight that meets ALL constraints” là hướng dẫn cần thiết nhưng không phải cơ chế đảm bảo. Việc model trong case fail vẫn cố book VJ604 chứng minh kiểm tra bằng code là cần thiết.

8. FAILURE MODES VÀ BÀI HỌC THIẾT KẾ

8.1. Lặp vô hạn

Nếu tool trả lỗi mơ hồ “not found” và model lặp cùng input sai định dạng 07/10, agent có thể tiêu tốn nhiều lượt mà không tiến triển. File failure mode dừng agent khi một cặp (tool, args) đã xuất hiện ba lần. `ModelCallLimitMiddleware` là giới hạn bổ sung để không vượt quá ngân sách model call.

8.2. Bịa dữ liệu từ tool

Ở mode hallucination, model báo đã đặt VN999 với giá 1.200.000 VND dù không tool nào trả dữ liệu đó. Hàm `ungrounded_facts()` trích flight code và giá từ final answer, sau đó đối chiếu với LOG. Trong nghiệp vụ thật, cách kiểm tra nên dựa trên schema và dữ liệu có nguồn gốc rõ ràng thay vì chỉ regex.

8.3. Trôi mục tiêu

Model có thể chuyển tối ưu từ “đúng mọi ràng buộc” sang “chuyển rẻ nhất”, nên chọn QH118 dù bay sau 12:00. `check_permission()` đặt kiểm tra ngay trước `book_seat`, thời điểm cuối cùng trước khi thay đổi booking. Mọi nhánh agent đều dùng kiểm tra này.

8.4. Lỗi trạng thái

Kết quả rỗng `{}` từ `search_flights` trong mode state biểu diễn lỗi dịch vụ hoặc timeout, không có nghĩa là dữ liệu nghiệp vụ xác nhận không có chuyến bay. `is_empty()` phân biệt hai tình huống này để agent handoff “retry later” thay vì kết luận sai cho người dùng.

9. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

9.1. Hạn chế

- Model giả phát lại đúng kịch bản đã lập trình; không phản ánh biến thiên câu trả lời hay khả năng lập luận của model thật.
- Tập dữ liệu chỉ có vài chuyến bay, không bao phủ hãng bay, ghế trống, hạng vé, hoàn/hủy và thay đổi giá theo thời gian.
- Mock database nằm trong bộ nhớ, mất trạng thái khi chương trình kết thúc và không xử lý concurrency hay idempotency.
- Thanh toán không có xác nhận người dùng, không xử lý lỗi thanh toán, refund hay giao dịch hai pha.
- Bảng kết quả chưa có token, chi phí hoặc latency; do đó không dùng để so sánh hiệu năng production.

9.2. Hướng phát triển

- Thay `GenericFakeChatModel` bằng `ChatGoogleGenerativeAI` hoặc một chat model thật, giữ nguyên harness để so sánh công bằng.
- Viết tập test cố định gồm nhiều mức giá, giờ bay, lỗi tool và lựa chọn thay thế; lặp mỗi mẫu nhiều lần.
- Ghi các chỉ số success rate, số tool/model call, số action bị chặn, handoff, token, chi phí và latency.

- Cài workflow trực tiếp bằng LangGraph StateGraph để thể hiện node search, plan, approval, execute, recover và handoff rõ ràng hơn.
- Tích hợp API thực trong môi trường sandbox, thêm xác thực, audit log, xác nhận thanh toán của người dùng và cơ chế bù trừ khi lỗi.

10. KẾT LUẬN

Bài làm đã xây dựng agent đặt vé máy bay giả lập bằng LangChain với bốn tool nghiệp vụ: tìm chuyến, giữ chỗ, thanh toán và đọc booking. Phần harness đáp ứng bốn yêu cầu chính của đề: lưu ràng buộc dưới dạng dữ liệu, kiểm tra quyền trước hành động, xác minh hoàn thành bằng code và bàn giao khi agent không thể tiếp tục.

Ba mẫu ReAct, Plan-then-Execute và Lai đều hoạt động trên cùng tư duy kiểm soát. ReAct linh hoạt theo từng observation; Plan-then-Execute giúp người dùng xem kế hoạch trước khi hành động; mẫu Lai dùng plan để tạo minh bạch nhưng vẫn có khả năng phục hồi bằng ReAct. Các demo failure mode cho thấy tính cần thiết của harness: model không nên là thành phần duy nhất quyết định an toàn hoặc sự hoàn thành của nghiệp vụ.

Vì model và dữ liệu đều là giả lập, báo cáo không kết luận về hiệu năng LLM thật. Tuy nhiên, thiết kế hiện tại tạo nền tảng rõ ràng để thay model giả bằng API thật, mở rộng dữ liệu và thực hiện benchmark có kiểm soát ở bước tiếp theo.

Link github: <https://github.com/BinCry/BTVN-Agentic-AI/tree/main/BTVN03>

TÀI LIỆU THAM KHẢO

[1] LangChain. LangChain overview.

<https://docs.langchain.com/oss/python/langchain/overview>

[2] LangChain. LangGraph overview.

<https://docs.langchain.com/oss/python/langgraph/overview>

[3] Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. <https://arxiv.org/abs/2210.03629>

[4] Pydantic. Documentation. <https://docs.pydantic.dev/>